TableJsonParse

№ id	43
チェック	☐
機能カテゴリー	テキスト処理
機能概要	テーブル内JSONをパース

１．この機能でできること

CSVテーブルの中にあるJSONデータから、指定したパスの値を抽出して新しい列として追加できます。

できること：

- CSVの特定列に含まれるJSONから値を抽出
- 抽出した値を新しい列としてテーブルに追加
- ネストされたJSONの深い階層からも抽出可能
- CSV内の全行に対して一括処理

こんな場面で使えます：

- LLMの構造化出力結果（JSON列）から特定の値を取り出したいとき
- APIレスポンスがJSONで格納されている列から必要なデータを抽出したいとき
- テーブルのJSON列を展開して分析しやすくしたいとき

必要なもの：

- CSV文字列 `型：文字列` （例：JSON列を含むCSVテキスト）

出てくるもの：

- 抽出列が追加されたCSV `型：文字列` （例：新しい列が追加されたCSVテキスト）

設定する項目：

- `extract_path`：抽出パス（例："列名.jsonパス"。ドットの前が列名、後がJSONパス）

２．基本の使い方

使用例1: JSON列から値を抽出

やりたいこと： result列のJSONからnameを抽出

```
Input(
    input_csv="id,result\n1,'{"name": "田中", "age": 30}'\n
2,'{"name": "佐藤", "age": 25}'"  # 型: 文字列
)

Params(
    extract_path="result.name"  # 型: 文字列 | result列のJSON
からnameを抽出
)

# 結果（result.name列が追加）
Output(
    output_csv="id,result,result.name\n1,'{...}',田中\n
2,'{...}',佐藤"  # 型: 文字列
)
```

使用例2: ネストされたJSONから抽出

やりたいこと： 深い階層の値を取得

```
Input(
    input_csv="id,data\n1,'{"info": {"company": "ABC株式会
社"}}'"  # 型: 文字列
)

Params(
    extract_path="data.info.company"  # 型: 文字列 | 深い階層
を指定
)

Output(
    output_csv="id,data,data.info.company\n1,'{...}',ABC株
式会社"  # 型: 文字列
)
```

３．他の機能との組み合わせ活用と注意点

よく使う組み合わせ

1. LLMの構造化出力を展開

流れ： StructuredOutput → TableJsonParse

- **やること：** LLMの構造化出力（JSON列）から必要なフィールドを抽出して列追加

2. API結果の解析

流れ： Run DI → TableJsonParse → FilterColumnValue

- **やること：** API結果のJSONから値を抽出し、条件でフィルタリング

3. 複数フィールドの抽出

流れ： TableJsonParse → TableJsonParse

- **やること：** 同じJSON列から複数のフィールドを順番に抽出

気をつけること

⚠️ extract_pathの書式

- 最初のドットの前が**列名**、後が**JSONパス**です
- 例: "result.name" → result列のJSONからnameを抽出
- 例: "data.info.company" → data列のJSONからinfo.companyを抽出

💡 同名列がある場合

- 抽出先の列名が既存列と重複する場合、"_1"が自動で付きます

✅ エラーが出たら

- 空の結果 → extract_pathの列名が正しいか確認
- JSONパースエラー → 列の中身が有効なJSONか確認